PLATE & PANTRY

Restaurant Ordering & Operations Platform

A full-stack restaurant experience for menu discovery, persistent cart management, secure guest checkout, order tracking, inventory control, and restaurant administration.

REACT	**TYPESCRIPT**	**JAVA 21**
SPRING BOOT	**POSTGRESQL**	**DOCKER**

	foodorder-production-ec70.up.railway.app
	github.com/AshenRandira/Foodorder

PROJECT DOCUMENTATION

Prepared by Ashen Randira

Live links

- Live application: foodorder-production-ec70.up.railway.app
- Source repository: github.com/AshenRandira/Foodorder
- Admin login: foodorder-production-ec70.up.railway.app/admin/login

PayHere is disabled in the public demo until an approved domain and domain-specific Sandbox secret are available. The integration and callback-verification logic remain implemented and testable. No payment is simulated as successful when PayHere is unavailable.

Contents

- Features
- Technology stack
- Architecture
- Project structure
- Getting started
- Environment configuration
- Available commands
- API overview
- Order and payment design
- Security
- Testing
- Deployment
- Known limitations

Features

Customer experience

- Responsive, mobile-first restaurant storefront
- Featured dishes, cuisine categories, search, and menu filtering
- Product pages with price, availability, stock, and quantity controls
- Persistent cart backed by browser `localStorage`
- Backend-validated guest checkout and delivery information
- Server-calculated prices, delivery fee, line totals, and order total
- PayHere Sandbox checkout with backend-generated hashes
- WhatsApp ordering with a backend-generated, URL-encoded message
- Private order tracking through an unguessable access token

- Clear loading, empty, unavailable, sold-out, validation, and error states

Restaurant administration

- Session-based administrator authentication
- Dashboard for orders, sales, kitchen queue, and low-stock items
- Category creation, visibility, ordering, and editing
- Product creation, editing, featuring, restocking, and deactivation
- Order search and filtering
- Customer, delivery, item, payment, and fulfilment views
- Server-controlled fulfilment status transitions
- Stock restoration for eligible cancellations and failed payments

Technology stack

Area	Technologies
Frontend	React 19, TypeScript 7, Vite 8, Tailwind CSS 4
Routing and data	React Router 7, Axios, React Context API, `localStorage`
Interface	Lucide React icons, responsive CSS, DM Sans and Fraunces typography
Backend	Java 21, Spring Boot 4.1, Spring MVC, Spring Data JPA
Security	Spring Security, server sessions, BCrypt, CSRF protection, CORS
Database	PostgreSQL 17, Hibernate ORM, Flyway migrations, HikariCP
Payments and ordering	PayHere Sandbox Checkout API, WhatsApp `wa.me` integration
Testing	JUnit 5, Mockito, MockMvc, Vitest, Testing Library, H2 test database
Tooling and delivery	Maven, npm, Docker, Docker Compose, Nginx, Railway, Neon

A reusable written description of the stack is available in TECHNOLOGIES.md.

Architecture

Production

```
Browser
   |
   | HTTPS: React routes and /api requests
   v
Railway single-container service
   |
   |-- React production assets
   |-- Spring Boot REST API
   |-- session and CSRF security
   |-- PayHere callback verification
   |
   v
Neon PostgreSQL
```

Local Docker Compose

```
Browser
   |
   v
Nginx frontend :5173
   |
   | /api proxy
   v
Spring Boot :8080
   |
   v
PostgreSQL :5432
```

The backend follows a layered structure: controllers receive HTTP requests, services enforce business rules and transactions, repositories handle persistence, DTOs define external contracts, and domain entities model the database. Prices and totals supplied by the browser are never trusted.

Project structure

```
Foodorder/
|-- backend/
|   |-- src/main/java/com/plateandpantry/
|   |   |-- config/          Security and application configuration
|   |   |-- controller/      REST endpoints
|   |   |-- domain/          JPA entities and enums
|   |   |-- dto/             Request and response models
|   |   |-- error/           API exception handling
|   |   |-- repository/      Spring Data repositories
|   |   `-- service/         Business and transactional logic
|   `-- src/main/resources/
|       `-- db/migration/    Flyway schema and seed data
|-- frontend/
|   `-- src/
|       |-- admin/           Administration interface
|       |-- components/      Shared UI components
|       |-- context/         Cart and application state
|       |-- lib/             API client and helpers
|       |-- pages/           Customer-facing routes
|       `-- types/           TypeScript contracts
|-- scripts/                 Local startup helpers
|-- docker-compose.yml       Local three-service stack
`-- Dockerfile               Railway production image
```

Getting started

Prerequisites

- Docker Desktop with Docker Compose
- Or Java 21, Maven 3.9+, Node.js 24+, npm, and PostgreSQL 17
- Git

Option 1: Docker Compose

This is the simplest local workflow.

```
git clone https://github.com/AshenRandira/Foodorder.git
cd Foodorder
Copy-Item .env.example .env
```

Update the ignored `.env` with a strong PostgreSQL password, an administrator password of at least 12 characters, and a valid international WhatsApp number. Then run:

```
docker compose up --build
```

Open:

- Storefront: `http://localhost:5173`
- Admin: `http://localhost:5173/admin/login`
- API: `http://localhost:8080/api`

Stop the application without deleting PostgreSQL data:

```
docker compose down
```

Option 2: Local development

Start PostgreSQL:

```
docker compose up -d db
```

Start the backend in one terminal:

```
.\scripts\start-backend.ps1
```

Start the frontend in another terminal:

```
cd frontend
npm.cmd ci
npm.cmd run dev
```

The frontend development server runs at `http://localhost:5173` and proxies `/api` to `http://localhost:8080` by default. To preview the local frontend against another backend, set `VITE_API_URL=/api` and `VITE_PROXY_TARGET` only for the local Vite process.

Flyway creates and validates the database schema and seeds the demonstration menu on first startup. Do not delete the Docker volume unless you intentionally want to remove local database data.

Environment configuration

Real credentials belong only in the ignored local `.env` or the hosting platform's secret manager. Never commit them and never expose backend credentials through a `VITE_` variable.

Core variables

Variable	Purpose	Local example
`POSTGRES_DB`	Local Compose database name	`plate_pantry`
`POSTGRES_USER`	Local Compose database user	`plate_pantry`
`POSTGRES_PASSWORD`	Local Compose database password	Strong private value
`DB_URL`	Spring Boot JDBC connection	`jdbc:postgresql://localhost:5432/plate_pantry`
`DB_USERNAME`	Spring datasource user	Private value
`DB_PASSWORD`	Spring datasource password	Private value
`ADMIN_USERNAME`	Initial administrator username	`admin`
`ADMIN_PASSWORD`	Initial administrator password	At least 12 characters
`FRONTEND_ORIGINS`	Exact comma-separated allowed browser origins	`http://localhost:5173`
`SESSION_COOKIE_SECURE`	Restrict the session cookie to HTTPS	`false` locally, `true` in production
`SESSION_COOKIE_SAME_SITE`	Session cookie same-site policy	`lax`
`DELIVERY_FEE_LKR`	Backend-owned delivery fee	`350.00`
`WHATSAPP_BUSINESS_NUMBER`	International digits-only WhatsApp number	`94770000000`

PayHere variables

Variable	Purpose
`PAYHERE_ENABLED`	Enables checkout only when all Sandbox configuration is ready
`PAYHERE_MERCHANT_ID`	PayHere Sandbox merchant identifier
`PAYHERE_MERCHANT_SECRET`	Domain-specific merchant secret; backend only
`PAYHERE_CHECKOUT_URL`	Sandbox checkout endpoint
`PAYHERE_NOTIFY_URL`	Public server callback ending in `/api/payments/payhere/notify`
`PAYHERE_RETURN_URL`	Browser return route ending in `/order/return`
`PAYHERE_CANCEL_URL`	Browser cancellation route ending in `/checkout`

Configuration loading rules

- Docker Compose loads the root `.env` and maps `POSTGRES_*` values to the backend's `DB_*` variables.
- Spring Boot does not automatically load the root `.env`. Use operating-system variables or `scripts/start-backend.ps1`.
- Vite reads `frontend/.env` and exposes only variables beginning with `VITE_`.
- Railway does not receive the ignored local `.env`; configure each required variable in the Railway service.
- Neon CLI variables such as `DATABASE_URL`, `DATABASE_URL_UNPOOLED`, and `NEON_BRANCH` support local tooling and are not read directly by the Spring application.

See .env.example for the complete safe template.

Available commands

Frontend

```
cd frontend
npm.cmd run dev       # Start the Vite development server
npm.cmd test          # Run Vitest tests
npm.cmd run build     # Type-check and create the production bundle
npm.cmd run preview   # Preview the production bundle
```

Backend

```
cd backend
mvn spring-boot:run   # Start Spring Boot using process environment variables
mvn test              # Run backend tests
mvn package           # Build the executable JAR
```

Complete stack

```
docker compose up --build
docker compose ps
docker compose down
```

API overview

Method	Endpoint	Access
GET	/api/menu/categories	Public
GET	/api/menu/products	Public
GET	/api/menu/products/{id}	Public
POST	/api/orders	Public; requires Idempotency-Key
GET	/api/orders/{reference}?token=...	Private order token
GET	/api/config	Public frontend configuration
POST	/api/payments/payhere/notify	Public provider callback with signature verification
GET	/api/csrf	Public CSRF bootstrap
POST	/api/admin/auth/login	Public login endpoint
POST	/api/admin/auth/logout	Authenticated and CSRF protected
GET	/api/admin/dashboard	Administrator session
GET/POST/PUT/PATCH	/api/admin/categories, /api/admin/products	Administrator session and CSRF for mutations
GET/PATCH	/api/admin/orders	Administrator session and CSRF for mutations

Order and payment design

1. The frontend sends product identifiers and quantities, not trusted prices.
2. The backend locks products, checks availability and stock, and calculates all monetary values with `BigDecimal`.

3. An `Idempotency-Key` and database constraint prevent retrying checkout from creating duplicate orders or deducting stock twice.
4. Stock is reserved when a PayHere or WhatsApp order is persisted.
5. PayHere generates a redirect request using a backend-created hash.
6. Only a valid server callback can mark a PayHere order as paid. The browser return page cannot change payment status.
7. The callback handler verifies the signature, merchant, order reference, amount, and currency.
8. Callback fingerprints make repeated provider notifications idempotent.
9. Verified failure or cancellation releases stock exactly once.
10. WhatsApp orders remain awaiting confirmation until an administrator accepts them.

Security

- Server-side administrator sessions with `HttpOnly` cookies
- Secure cookies in deployed HTTPS environments
- BCrypt password hashing with cost 12
- CSRF cookie/header validation for authenticated mutations
- Exact-origin credentialed CORS; wildcard origins are not used
- Role protection for administration endpoints
- Login throttling after repeated failures
- Jakarta Bean Validation for API requests
- Token-protected customer order lookup
- Backend-owned totals and inventory validation
- PayHere callback signature and value verification
- Secrets excluded from Git, Docker build context, frontend bundles, and API responses
- Safe API errors without stack traces or credentials

Testing

Run all backend tests:

```
cd backend
mvn test
```

Run frontend tests and a production build:

```
cd frontend
npm.cmd test
npm.cmd run build
```

The automated suite covers checkout calculations, inventory validation, idempotency, status transitions, PayHere signature and value checks, duplicate callbacks, cancellation and failure behavior, admin authorization, and CSRF enforcement. Frontend tests cover core rendering and customer interactions.

Before a release, also verify:

- Responsive layouts at mobile, tablet, and desktop widths
- Menu, cart, checkout, and order tracking
- Administrator login and protected routes
- Direct navigation to React routes
- Database migrations against a clean test database
- A real PayHere Sandbox callback before claiming end-to-end payment success

Deployment

The root Dockerfile performs a multi-stage production build:

1. Node builds the React application with `VITE_API_URL=/api`.
2. Maven packages the Spring Boot application and includes the frontend assets.
3. A small Java 21 runtime image starts the complete application on Railway's `PORT`.

For Railway:

- Deploy from the repository root and let Railway detect the root Dockerfile.
- Use a Neon pooled JDBC hostname for `DB_URL`.
- Use the Neon direct hostname for `FLYWAY_URL`.
- Retain `sslmode=require` for both database connections.
- Set `FRONTEND_ORIGINS` to the exact Railway HTTPS origin.
- Set `SESSION_COOKIE_SECURE=true` and `SESSION_COOKIE_SAME_SITE=lax`.
- Configure the health check as `/api/menu/categories`.
- Keep PayHere disabled until the public domain is registered and the matching secret and callback URLs are configured.
- Keep the application at one replica while sessions remain in application memory.

Known limitations

- PayHere requires an accepted registered domain; temporary Cloudflare subdomains may be rejected.
- The public demonstration currently keeps PayHere disabled and leaves WhatsApp ordering available.
- Customer accounts, discounts, taxes, refunds, live driver tracking, and automatic reservation expiry are outside the current scope.
- Paid-order cancellation records the fulfilment result but does not automatically call a refund API.
- Horizontal scaling requires shared Spring Session storage or sticky sessions.
- Seed images are delivered from external image URLs and therefore require internet access.

Author

Developed by Ashen Randira.

License

This repository was created as a software engineering assessment and portfolio project. Add an explicit open-source license before redistributing or accepting third-party contributions.

Demo administrator

Public credentials for evaluating the administration workflow.

USERNAME	demo
PASSWORD	PlatePantryDemo2026!

How to use it

1. Open the deployed administration login page.
2. Select Use demo credentials to fill both fields automatically.
3. Sign in to access the dashboard, catalogue, inventory, and orders.

OPEN ADMIN LOGIN

This is a full-access administrator created for a dummy portfolio environment.

Disable DEMO_ADMIN_ENABLED before using the application with real customer data.